

**FEDERAL STATE AUTONOMOUS EDUCATIONAL INSTITUTION
FOR HIGHER EDUCATION
NATIONAL RESEARCH UNIVERSITY
«HIGHER SCHOOL OF ECONOMICS»**

Faculty of Computer Science

Ivan Nikulin / Никулин Иван

Name, Surname

Согласование позиционного кодирования для повторного
использования KV-кэша в многодокументных задачах

Russian title

Positional Encoding Alignment for Multi-Document KV Reuse

English title

Term paper

Field of study 01.04.02 «Applied Mathematics and Informatics»

Program: «Data Science»

Student

Ivan Nikulin

Supervisor

Alan Tomat

Moscow, 2026

Table of Contents

Introduction	2
1 Background and Related Work	4
1.1 Transformer inference and KV-cache reuse	4
1.2 Rotary Positional Embeddings	4
1.3 Relation to CacheBlend and RAG cache fusion	4
2 Methodology	6
2.1 Problem formulation	6
2.2 Positional Encoding Alignment	6
2.3 Compared strategies	7
2.4 Algorithm	7
3 Implementation Details	8
4 Experimental Setup	10
4.1 Model and hardware	10
4.2 Datasets	10
4.3 Metrics	11
5 Mathematical and Numerical Validation	12
5.1 Unit-level RoPE validation	12
5.2 Numerical anomaly during generation	12
5.3 Source of overflow and selective upcasting	13
6 Results and Discussion	14
6.1 Main results on MuSiQue and SAMSum	14
6.2 Main positive result on RULER	15
6.3 Why Baseline is still much better	16
6.4 Interpretation by task type	16
7 Limitations and Future Work	18
Conclusion	20
List of References	21

Introduction

Modern large language models are increasingly used as components of systems that retrieve, combine and reason over external knowledge. One of the most common practical settings is Retrieval-Augmented Generation, where a model receives a question together with retrieved documents and produces an answer grounded in that external context [1]. This reduces reliance on parametric memory, but it creates a computational bottleneck: as more documents are added to the prompt, the prefill stage becomes more expensive.

Autoregressive transformer inference can be divided into two stages. During prefill, the model processes the whole input prompt and builds the KV-cache for all transformer layers. During decoding, newly generated tokens reuse this cache, so the model does not recompute keys and values for old tokens at every generation step. This mechanism is very efficient for prefix reuse, but it is less straightforward for multi-document prompts where the same document can appear after different prefixes or at different positions in the context.

The difficulty is especially important for models that use Rotary Positional Embeddings. In RoPE-based architectures, positional information is not stored as a separate learned vector added to token embeddings. Instead, it is encoded through rotations applied to query and key representations [2]. Therefore, cached keys are not position-independent: the key for the same token is different when the token is processed at position 0 and when it is processed at position 3000. If a document is cached independently from position 0 and later inserted after a system prompt and several previous documents, direct cache concatenation breaks the positional geometry expected by the model.

This project studies Positional Encoding Alignment, abbreviated as PEA. In the code and result tables, the main strategy is called Aligned. The method independently precomputes document KV-caches, but before concatenating them into a global DynamicCache, it rotates each segment key tensor by the RoPE phase corresponding to the segment offset in the final prompt. The values are not rotated, because values represent content vectors aggregated by attention weights rather than vectors used in query-key positional scoring.

The project has three main goals. The first goal is engineering: to implement a modular pipeline that can precompute segment caches, transform them and assemble a valid Hugging Face DynamicCache. The second goal is methodological: to test whether RoPE-aware alignment improves segmented cache reuse compared with naive concatenation. The third goal is diagnostic: to understand why the method succeeds or fails on different task types and to separate true method limitations from numerical artifacts caused by fp16 inference on available hardware.

The key finding is that the hypothesis is partially confirmed. RoPE-aware alignment is clearly useful in synthetic retrieval tasks such as RULER [3], where the model must extract a hidden value from a long context. However, it is not sufficient to repro-

duce full-prefill behavior on more complex multi-hop reasoning and summarization tasks. The remaining gap is not primarily caused by a wrong RoPE implementation. It is caused by a deeper limitation of independent segment precomputation: each segment is encoded without inter-segment self-attention to the previous context.

1 Background and Related Work

1.1 Transformer inference and KV-cache reuse

The Transformer architecture is based on self-attention, where each token forms query, key and value representations and attends to other tokens in the sequence [4]. In standard full attention, the computational and memory costs grow quickly with sequence length. Optimized kernels such as FlashAttention reduce the practical cost of exact attention through IO-aware computation [5], but they do not remove the need to process long contexts during prefill.

KV-cache reuse is a natural way to reduce repeated computation. If the same prefix is used across several generations, the model can reuse already computed keys and values for that prefix. In a document-heavy RAG setting, however, reuse is more difficult. The same document may be reused across many questions, but it may be preceded by different system prompts, different retrieved documents or different instructions. A cache computed for one position is not automatically valid at another position when the model uses positional mechanisms such as RoPE.

1.2 Rotary Positional Embeddings

RoPE encodes token positions by rotating query and key vectors in two dimensional subspaces [2]. If a pair of coordinates is represented as a complex number, the rotation for position m can be written as:

$$\text{RoPE}_m(z_j) = z_j \cdot \exp(i \cdot m \cdot \theta_j). \quad (1.1)$$

The important property for this project is the linearity of phase shifts. A representation already rotated for position m can be shifted to position $m + \Delta$ by applying one additional rotation:

$$\text{RoPE}_{m+\Delta}(z_j) = \text{RoPE}_m(z_j) \cdot \exp(i \cdot \Delta \cdot \theta_j). \quad (1.2)$$

This property makes it possible to transform cached keys after they have already been computed. If a document was cached from position 0, but in the assembled prompt it should start at offset Δ , the key tensor can be rotated by an additional phase Δ . This does not require re-running the whole document through the model, at least for the positional part of the key representation.

1.3 Relation to CacheBlend and RAG cache fusion

Recent work such as CacheBlend studies more general cache reuse and cached knowledge fusion in RAG settings [6]. Such approaches recognize that naive reuse

of cached documents is not enough, because a document encoded in isolation does not contain interactions with other documents or with the final question. This project focuses on a narrower but important part of that problem: the positional correction of RoPE-based cached keys. It does not implement selective recomputation or full cache blending, but the experimental results point toward those methods as a natural next step.

2 Methodology

2.1 Problem formulation

Let the final prompt consist of a prefix S , a sequence of documents $D_1, D_2, \dots, D_n$ and a question or suffix Q . In the standard full-prefill Baseline, the model receives one continuous prompt:

$$P = S \oplus D_1 \oplus D_2 \oplus \dots \oplus D_n \oplus Q. \quad (2.1)$$

In this setting, the token positions of document D_i depend on the lengths of all previous segments:

$$\text{pos}(D_i) = |S| + \sum_{j=1}^{i-1} |D_j| + [0, |D_i| - 1]. \quad (2.2)$$

With independent caching, each document is processed separately, usually from local positions 0, 1, 2, Therefore, its cached keys contain RoPE phases for local positions rather than global positions. Direct concatenation of such caches creates an internally inconsistent prompt representation.

2.2 Positional Encoding Alignment

PEA corrects the positional mismatch by applying an additional RoPE phase shift to the cached keys of each segment. If segment D_i should start at offset Δ_i in the assembled prompt, the transformation is:

$$K_i^{\text{aligned}} = R_{\Delta_i} K_i^{\text{cached}}. \quad (2.3)$$

In complex notation, this is equivalent to multiplying each coordinate pair by the additional phase factor:

$$K_{i,j}^{\text{aligned}} = K_{i,j}^{\text{cached}} \cdot \exp(i \cdot \Delta_i \cdot \theta_j). \quad (2.4)$$

The value tensor is intentionally left unchanged:

$$V_i^{\text{aligned}} = V_i^{\text{cached}}. \quad (2.5)$$

This distinction is essential. RoPE affects query-key dot products and therefore the attention distribution over positions. Values serve as content vectors aggregated after attention weights have already been computed. Rotating values would introduce an operation that is not part of the original RoPE mechanism.

Algorithm 1: PEA for document cache reuse**Input:** documents $D_1, \dots, D_n$, prefix S , question suffix Q , model M with RoPE.

1. Precompute KV-cache for S .
2. Precompute KV-cache for each D_i independently.
3. Initialize global cache with $\text{cache}(S)$.
4. Set current offset = $|S|$.
5. For each document D_i :
 - a. take cached K_i, V_i ;
 - b. rotate K_i by RoPE phase shift for current offset;
 - c. keep V_i unchanged;
 - d. concatenate shifted K_i and original V_i ;
 - e. update offset + $= |D_i|$.
6. Run generation on Q using assembled cache.

Output: generated answer.

Figure 1. Pseudocode of Positional Encoding Alignment.

2.3 Compared strategies

The experiments compare three strategies. Baseline is the standard full-prefill run over the complete prompt. It is the reference method and the upper bound for the segmented strategies. Aligned is the proposed PEA strategy: independent segment precompute, RoPE shift of cached keys according to the global offset and cache concatenation. Naive is an ablation control: it uses the same independent precompute and the same assembly procedure, but does not apply the RoPE shift. Therefore, the difference between Aligned and Naive isolates the contribution of positional alignment.

2.4 Algorithm

The algorithm is intentionally modular: the transformation function can be replaced. In the implementation, Aligned uses `shift_cache`, while Naive uses `identity_transform`.

3 Implementation Details

The implementation is organized as a compact experimental pipeline. The main entry point is `main_benchmark.py`. It loads the model and tokenizer, iterates through enabled datasets, prepares the prompt segments, runs the full Baseline, precomputes the segmented caches and then evaluates Aligned and Naive. Each case is stored with the target answer, generated outputs and computed metrics.

The most important implementation detail is the `shift_cache` function. It computes the RoPE inverse frequencies from the model configuration, builds cosine and sine tensors for the required offset, temporarily casts the key tensor to float32, applies the rotation and then casts the result back to the original dtype. The value tensor is returned without modification. The temporary float32 cast is used to reduce avoidable numerical error during the rotation.

The cache assembly logic is implemented through Hugging Face `DynamicCache`. The prefix cache is used as the initial cache. For every following segment, the selected transformation is applied, and the key and value tensors are concatenated along the sequence dimension for every layer. The resulting cache is then passed to generate together with the suffix or question tokens.

Eager attention is selected because it allows attention weights to be extracted for diagnostic metrics. The model used in the final experiments is Qwen/Qwen2.5-7B-Instruct [7].

Table 3.1

Main source files and their responsibilities

File	Role in the project
config.yaml	Stores model settings, dataset settings, seed, dtype, attention implementation and output paths.
main_benchmark.py	Runs the full experimental loop: Baseline, precompute, Aligned, Naive and CSV export.
src/utils_rope.py	Implements <code>rotate_half</code> , <code>shift_cache</code> and <code>identity_transform</code> . This is the core RoPE alignment module.
src/utils_cache.py	Extracts KV-cache lists, packs tensors into <code>DynamicCache</code> and assembles segmented caches.
src/utils_data.py	Builds dataset-specific prefix, segments, suffix and target answer for MuSiQue, SAMSum and RULER.
src/utils_metrics.py	Computes F1, Exact Match, Rouge-L, KL divergence, Top-1 Agreement, GT rank, NLL and attention correlation.
src/attention_patch.py	Applies selective upcasting in Qwen2 eager attention to avoid fp16 overflow in QK^T during decoding.
scripts/run_validation.py	Runs unit-level checks for reversibility, attention invariance and native consistency of RoPE shifting.

4 Experimental Setup

4.1 Model and hardware

All final experiments are conducted with Qwen/Qwen2.5-7B-Instruct. The model is loaded through Hugging Face Transformers with fp16 weights, eager attention and DynamicCache. The hardware environment is a single NVIDIA V100 GPU with 32 GB of memory. Because V100 does not support bfloat16, the project uses fp16 weights and an additional selective upcasting patch for numerically sensitive attention computations during decoding.

The use of fp16 is important for interpreting the results. fp16 has a maximum finite value of 65,504. In rare cases this caused the attention dot product to exceed the fp16 range, which is why a separate numerical stability section is included.

4.2 Datasets

The method is evaluated on three task families. They were selected to test different forms of long-context behavior: multi-hop reasoning, summarization and synthetic retrieval.

Table 4.1

Evaluation datasets			
Dataset	Samples	Task type	Target behavior tested
MuSiQue	2,000	Multi-hop QA	Reasoning over multiple paragraphs and combining evidence from different parts of the context.
SAMSum	1,000	Dialogue summarization	Compression of a dialogue into a short summary with global discourse understanding.
RULER 4k	500	Needle-in-a-Haystack QA	Exact retrieval of a hidden magic number from a 4k-token synthetic context.
RULER 8k	500	Needle-in-a-Haystack QA	Exact retrieval of a hidden magic number from an 8k-token synthetic context.

For RULER, the F1-score is computed at the character level. This is appropriate because the target output is a numeric code, and a character-level score captures partial extraction errors more informatively than token-level word overlap.

4.3 Metrics

The report uses two groups of metrics. The first group measures final output quality. For QA tasks, the main metrics are Exact Match and F1-score. For SAMSum, Rouge-L is used as the main summarization metric, while F1 is kept as an auxiliary overlap measure.

The second group measures behavioral and distributional similarity to the full-prefill Baseline. Top-1 Agreement measures how often the tested strategy chooses the same most likely first generated token as the Baseline. KL divergence measures the distance between the strategy distribution and the Baseline distribution over the top probability mass. NLL loss evaluates the confidence assigned to the first token of the ground-truth answer. GT Rank Median reports the median rank of the ground-truth first token. Attention Correlation compares attention weight patterns using Spearman correlation.

A limitation of these internal metrics is that they are local diagnostics. In the current implementation, KL divergence, NLL loss, GT rank and Top-1 Agreement are computed on the first generation step. They are useful for analyzing whether the tested strategy follows the Baseline at the start of decoding, but they should not be interpreted as full-sequence equivalence metrics.

5 Mathematical and Numerical Validation

5.1 Unit-level RoPE validation

Before the main experiments, the RoPE shift operation is validated independently. The validation consists of three checks. First, reversibility is tested by shifting a random float32 key tensor by $+\Delta$ and then by $-\Delta$. The maximum absolute reconstruction error is $4.77 \cdot 10^{-7}$. Second, attention invariance is tested by comparing an attention score before and after a consistent shift of both query and key. The observed difference is $6.10 \cdot 10^{-5}$. Third, native consistency is tested by comparing a manually shifted key tensor with a key tensor produced by the model when explicit shifted `position_ids` are supplied. The maximum absolute difference is 0.03085 and the cosine similarity is 1.0.

These checks show that the implemented rotation behaves as expected at the level of RoPE geometry. They do not prove that segmented-cache generation is equivalent to full-prefill generation, because full-prefill generation also includes inter-segment attention and layer-by-layer hidden-state interactions. However, they are sufficient to rule out a basic mathematical implementation error in the RoPE shift itself.

5.2 Numerical anomaly during generation

During early evaluation, the segmented strategies sometimes produced repetitive exclamation marks instead of factual answers. The issue was not a semantic failure of the method. A logit-level inspection showed that the probability distribution after softmax contained NaN values. In greedy decoding, argmax over such invalid values can collapse to token ID 0, which corresponds to the exclamation mark in the Qwen2.5 tokenizer.

The first hypothesis was that independent segmented precomputation corrupted the magnitude of the key tensors. To test this, problematic RULER examples were analyzed layer by layer and compared to the full-context Baseline cache. The analysis focused on cosine similarity, mean absolute error and maximum key activation differences.

Figure 2 shows that Aligned remains substantially closer to the full-context Baseline than Naive across the transformer layers. The cosine similarity of Aligned is close to 1.0 in early layers and remains much higher than Naive in deeper layers. MAE is also consistently lower for Aligned. The maximum key activation difference does not indicate a systematic amplitude explosion in the cache. This means that the repeated exclamation mark artifact was not caused by corrupted key magnitudes after the RoPE shift.

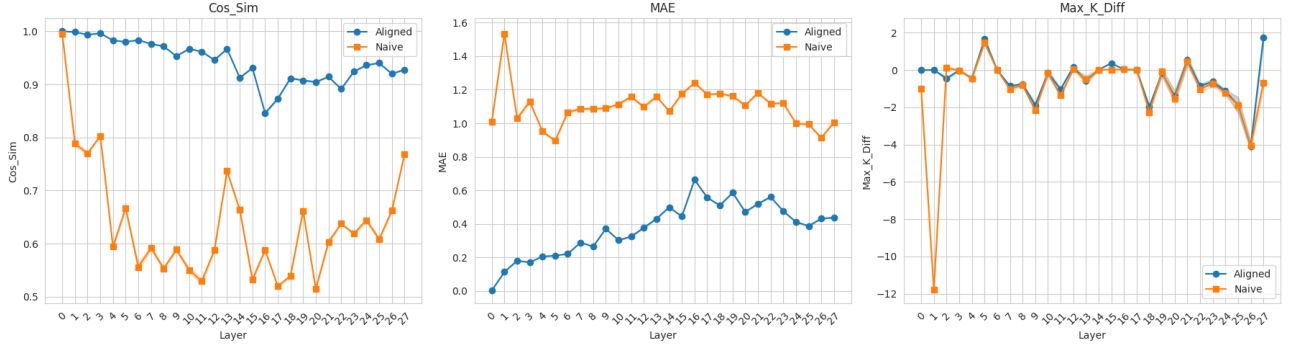


Figure 2. Layer-wise mathematical validation of K-cache integrity for Aligned and Naive strategies, averaged over context segments: (a) Cosine similarity relative to the full-context Baseline; (b) Mean Absolute Error (MAE) of cached values; (c) Deviation in maximum activation amplitudes ($\max|K|_{aligned} - \max|K|_{base}$).

5.3 Source of overflow and selective upcasting

The investigation then shifted to the attention computation itself. A teacher-forcing diagnostic loop monitored the scalar product QK^T during generation. The Baseline usually remained inside the safe fp16 range, while Aligned and Naive could occasionally cross the fp16 maximum finite value of 65,504 after a few decoding steps. Once the dot product overflowed to infinity, the following softmax could produce NaN values, which then caused invalid generation.

Because the available V100 GPU does not support bfloat16, the implemented mitigation was selective upcasting. The attention patch computes QK^T and softmax in float32 during decoding when a past cache is present, then returns the attention probabilities to the original dtype for the remaining operations. This is not the same as running the whole model in float32. It targets only the numerically sensitive operation, and the overhead is small because during decoding the query length is short.

Selective upcasting eliminated the observed exclamation mark artifacts and made the evaluation stable. Importantly, it did not make the segmented-cache strategies equivalent to the full Baseline. The remaining quality gap after upcasting is therefore interpreted as a methodological limitation of independent segmented caching, not as a hardware overflow artifact.

6 Results and Discussion

6.1 Main results on MuSiQue and SAMSum

The MuSiQue [8] and SAMSum [9] results show that RoPE alignment alone is not sufficient for complex generation tasks. On MuSiQue, Baseline remains much stronger than both segmented-cache strategies. Aligned has slightly lower F1 and EM than Naive, but it has better NLL loss and a slightly higher Top-1 Agreement. This suggests that the Aligned distribution can move closer to the correct first-token target even when the final generated answer does not improve under F1 or Exact Match.

Table 6.1

MuSiQue results, 2,000 validation samples

Strategy	F1	EM	NLL	KL	Attn Corr	GT Rank Med.	Top1 Agree
Baseline	0.3043	0.1549	–	–	–	–	–
Aligned	0.1101	0.0469	7.8528	4.5852	0.7597	20	0.1642
Naive	0.1154	0.0491	8.4519	4.5435	0.7606	22	0.1615

On SAMSum, the pattern is similar but less severe. Naive is slightly better than Aligned on F1 and Rouge-L, while Aligned has lower KL divergence. This is important because summarization metrics are sensitive to the exact wording of the generated text. A strategy can be closer to Baseline at the first-token distribution level and still produce a slightly worse final summary under Rouge-L.

Table 6.2

SAMSum results, 1,000 validation samples

Strategy	F1	Rouge-L	KL	Attn Corr	Top1 Agree
Baseline	0.3714	0.3229	–	–	–
Aligned	0.3174	0.2703	1.3114	0.6960	0.5055
Naive	0.3238	0.2804	1.4204	0.7482	0.5226

These results should not be interpreted as a complete failure of the method. Rather, they show that the method solves only one part of the problem. It fixes positional phase mismatch, but it does not reconstruct the cross-segment interactions needed for multi-hop reasoning or global summarization.

6.2 Main positive result on RULER

The strongest positive results appear on RULER. This is expected because RULER Needle-in-a-Haystack is primarily a retrieval task. The model must locate and reproduce a hidden magic number from a long context. In such a setting, correct positional geometry is especially important.

Table 6.3

RULER results, 500 samples for each context length

Dataset	Strategy	F1	EM	NLL	KL	GT Rank Med.	Top1 Agree
RULER 4k	Baseline	0.9860	0.9860	–	–	–	–
RULER 4k	Aligned	0.5665	0.5360	2.5228	2.5491	1	0.5960
RULER 4k	Naive	0.4297	0.3780	3.3855	3.3805	2	0.4640
RULER 8k	Baseline	1.0000	1.0000	–	–	–	–
RULER 8k	Aligned	0.6610	0.6380	1.6631	1.6609	1	0.6900
RULER 8k	Naive	0.4737	0.4200	2.9869	2.9867	1	0.5160

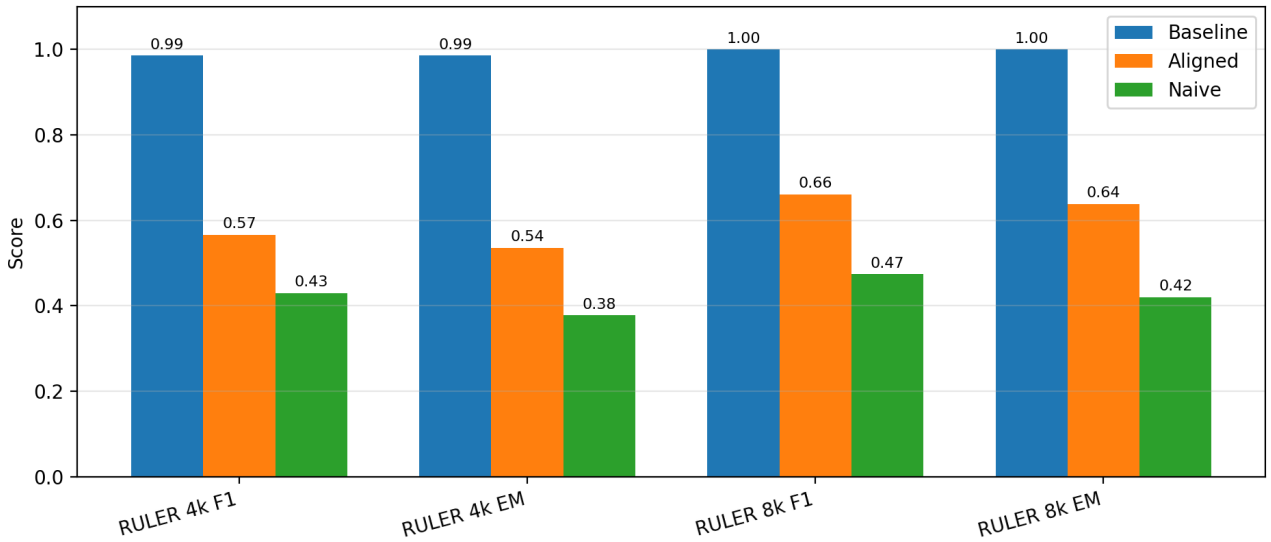


Figure 3. RULER retrieval quality for Baseline, Aligned and Naive strategies.

On RULER 4k, Aligned improves F1 from 0.4297 to 0.5665 compared with Naive, which is an approximate relative improvement of 31.8%. Exact Match improves from 0.3780 to 0.5360, or about 41.8%. NLL loss and KL divergence are also lower for Aligned, and Top-1 Agreement increases from 0.4640 to 0.5960.

On RULER 8k, the improvement is even clearer. Aligned improves F1 from 0.4737 to 0.6610, an approximate relative improvement of 39.5%. Exact Match in-

creases from 0.4200 to 0.6380, or about 51.9%. NLL loss is reduced from 2.9869 to 1.6631, and KL divergence is reduced from 2.9867 to 1.6609. These results show that the RoPE-aware offset correction makes the segmented cache substantially more usable for long-context retrieval than simple cache concatenation.

The fact that RULER 8k scores are higher than RULER 4k should not be interpreted as proof that the method improves monotonically with context length. The 4k and 8k subsets are separate synthetic samples and may differ in answer positions, distractors and prompt realizations. The robust conclusion is narrower and stronger: Aligned outperforms Naive on both RULER settings.

A second plausible explanation is connected with the fixed segmentation scheme used in the benchmark. The context is divided into the same number of segments, so the 8k setting contains longer segments than the 4k setting. With longer segments, more local context around the hidden number remains inside one independently pre-computed segment, while the relative share of boundaries between segments becomes smaller. This can reduce the practical impact of missing inter-segment self-attention and make the RULER 8k subset more favorable for the Aligned strategy. This explanation should be treated as a hypothesis rather than a proven causal claim, because the 4k and 8k samples also differ in their generated content and answer placement. The underlying limitation is discussed in more detail in the next section.

6.3 Why Baseline is still much better

Although Aligned improves over Naive on RULER, it remains below Baseline. This is expected. RoPE alignment corrects only the positional phase of the cached keys. It does not recreate the hidden states that would have been produced if the segment had been processed inside the full prompt.

In full-prefill inference, when document D_i is processed, its tokens can attend to the prefix and to previous documents. This influences hidden states in every subsequent layer. In independent precomputation, the document is processed alone, so its key and value representations are missing inter-segment self-attention effects. Even after the key tensor is shifted to the correct global offset, the content of K and V is still based on isolated segment processing.

This explains why the layer-wise validation can show strong K-cache improvement while the final generation still differs from Baseline. The Aligned cache is geometrically better than Naive, but it is not a full reconstruction of the full-context cache. The remaining error accumulates through layers and decoding steps.

6.4 Interpretation by task type

The results are consistent with the nature of each task. RULER is retrieval-oriented. The model mainly needs to attend to a precise location and copy a numeric answer. Therefore, correcting positional alignment has a direct effect. MuSiQue is

multi-hop QA. The model must combine evidence across multiple paragraphs, and those cross-paragraph relationships are precisely what independent segment precomputation cannot fully encode. SAMSum requires global summarization of a dialogue, including speaker continuity and discourse compression. This is also difficult to recover from independently cached fragments.

Therefore, the project should not be presented as a method that fully solves arbitrary segmented KV-cache reuse. A more accurate conclusion is that RoPE-aware alignment is a necessary and useful component for cache reuse, especially in retrieval-heavy long-context scenarios, but it is insufficient on its own for complex reasoning and summarization.

7 Limitations and Future Work

The main limitation is the loss of inter-segment self-attention. When each document is precomputed independently, the model cannot use previous documents or the full prompt while building that segment representation.

This limitation also explains why the exact segmentation scheme matters. If the number of segments is fixed, increasing the total context length increases the average segment length. Longer segments preserve more within-segment self-attention during independent precomputation and reduce the relative frequency of segment boundaries. As a result, some retrieval tasks may become less sensitive to the missing cross-segment recomputation, even when the total prompt is longer. For this reason, the comparison between 4k and 8k RULER settings should be interpreted jointly with the segmentation setup rather than as a simple monotonic effect of context length.

The second limitation is that the method shifts only K . This is correct with respect to RoPE, but it also means that the method intentionally does not modify V or hidden states. If the values themselves differ from full-prefill values because the segment was encoded without previous context, the method has no mechanism to repair that difference.

The third limitation concerns the metrics. The internal distributional metrics are valuable diagnostics, but several of them are computed at the first generation step. They should be interpreted as local indicators of behavioral similarity to Baseline rather than as complete sequence-level equivalence. This matters especially for summarization, where a good final answer depends on many generated tokens.

The fourth limitation is hardware-specific numerical behavior. The V100 environment required fp16 inference and did not allow bfloat16. The selective upcasting patch solved the observed overflow problem, but results on newer GPUs with native bfloat16 could be numerically cleaner. At the same time, this does not invalidate the main methodological conclusion, because the post-upcasting gap to Baseline remains explainable by missing inter-segment interactions.

This limitation is closely related to the motivation behind CacheBlend [6]. CacheBlend observes that precomputed KV-caches of reused text chunks cannot always be directly inserted into a new prompt, because independently cached chunks do not incorporate attention interactions with the preceding text. Its solution is to selectively recompute the KV states of a small subset of tokens, partially recovering the missing context-dependent interactions while still avoiding full prefill. In this sense, PEA can be viewed as a cheap and mathematically clean positional correction layer: it fixes the RoPE phase mismatch, but it does not solve the deeper problem of missing inter-segment self-attention. Therefore, a natural next step is to combine PEA with CacheBlend-like selective recomputation.

Future experiments should also include direct latency and memory measurements. The current work focuses on correctness, quality and diagnostics. Since the motivation for KV-cache reuse is computational efficiency, a complete system-level

evaluation should report prefill time, decode time, cache storage cost and the break-even point where precomputing and storing document caches becomes beneficial.

Conclusion

This project implemented and evaluated Positional Encoding Alignment for RoPE-aware reuse of independently precomputed KV-caches. The method rotates cached key tensors according to their global offsets before assembling them into a shared DynamicCache. The implementation was validated with reversibility, attention invariance and native consistency checks, and a separate numerical stability investigation identified and resolved fp16 overflow in the attention dot product through selective upcasting.

The experimental results partially confirm the hypothesis. Aligned strongly outperforms Naive on RULER 4k and 8k, showing that positional correction is important for long-context retrieval. At the same time, Aligned does not fully recover Baseline behavior and does not consistently improve external quality metrics on MuSiQue and SAMSum. This shows that RoPE alignment solves only the positional part of segmented-cache reuse.

The central conclusion is that RoPE-aware alignment is necessary but not sufficient. It repairs the geometric inconsistency introduced by naive cache concatenation, but it cannot reconstruct inter-segment self-attention that was absent during independent precomputation. Therefore, the method is most promising as a component of a larger cache reuse pipeline, especially one that combines positional correction with selective recomputation or cache blending.

From an engineering perspective, the project also demonstrates that careful numerical diagnostics are essential for long-context LLM experiments. Without identifying the fp16 overflow issue, the observed generation failures could have been incorrectly attributed to the method itself. After selective upcasting, the project provides a clearer and more reliable picture: the method is technically valid, useful in retrieval scenarios and limited by a well-defined modeling constraint.

List of References

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.
- [2] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2021. arXiv: 2104.09864 [cs.CL].
- [3] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. *RULER: What’s the Real Context Size of Your Long-Context Language Models?* 2024. arXiv: 2404.06654 [cs.CL].
- [4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.
- [5] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 16344–16359.
- [6] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. “CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion”. In: *Proceedings of the Twentieth European Conference on Computer Systems*. 2025, pp. 94–109.
- [7] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. *Qwen2.5 Technical Report*. 2024. arXiv: 2412.15115 [cs.CL].
- [8] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. “MuSiQue: Multihop Questions via Single-hop Question Composition”. In: *Transactions of the Association for Computational Linguistics* 10 (2022), pp. 539–554. DOI: 10.1162/tacl_a_00475.

- [9] Bogdan Gliwa, Iwona Mochol, Maciej Biesek, and Aleksander Wawer. “SAMSum Corpus: A Human-Annotated Dialogue Dataset for Abstractive Summarization”. In: *Proceedings of the 2nd Workshop on New Frontiers in Summarization*. 2019, pp. 70–79.